

CSA0613 - Design and Analysis of Algorithms

CO2 ASSESSMENT TOOL 2: SCENARIO-BASED

Implementation of the Brute Force Closest Pair Algorithm for Delivery Locations

Aim

To implement the Brute Force Closest Pair Algorithm to find the closest pair of delivery locations and analyse its performance.

Problem Statement

A delivery company maintains GPS coordinates of delivery points and computes distances between every pair to find the closest drop location. As the number of locations increases, delays are observed.

Objectives

- Implement the brute-force algorithm.
- Compute Euclidean distances.
- Find the closest pair.
- Analyse time complexity and performance.

Algorithm

1. Input locations.
2. Set minimum distance to infinity.
3. Compare every pair.
4. Compute Euclidean distance.
5. Update minimum distance if required.
6. Display closest pair and minimum distance.

Python Program

```
import math

# Function to calculate Euclidean distance
def distance(p1, p2):
    return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

# Brute Force Closest Pair Algorithm
def closest_pair(points):
    n = len(points)
    min_dist = float('inf')
    closest = None

    for i in range(n):
        for j in range(i + 1, n):
            d = distance(points[i], points[j])

            print(f"Distance between {points[i]} and {points[j]} = {d:.2f}")

            if d < min_dist:
                min_dist = d
                closest = (points[i], points[j])

    return closest, min_dist

# Sample GPS coordinates (x, y)
locations = [
    (2, 3),
    (12, 30),
    (40, 50),
    (5, 1),
    (12, 10),
    (3, 4)
]

pair, dist = closest_pair(locations)

print("\nClosest Pair:", pair)
print("Minimum Distance:", round(dist, 2))
```

Output

```
Distance between (2, 3) and (12, 30) = 28.79
Distance between (2, 3) and (40, 50) = 60.44
Distance between (2, 3) and (5, 1) = 3.61
Distance between (2, 3) and (12, 10) = 12.21
Distance between (2, 3) and (3, 4) = 1.41
Distance between (12, 30) and (40, 50) = 34.41
Distance between (12, 30) and (5, 1) = 29.83
Distance between (12, 30) and (12, 10) = 20.00
Distance between (12, 30) and (3, 4) = 27.51
Distance between (40, 50) and (5, 1) = 60.22
Distance between (40, 50) and (12, 10) = 48.83
Distance between (40, 50) and (3, 4) = 59.03
Distance between (5, 1) and (12, 10) = 11.40
Distance between (5, 1) and (3, 4) = 3.61
Distance between (12, 10) and (3, 4) = 10.82

Closest Pair: ((2, 3), (3, 4))
Minimum Distance: 1.41
```

Time Complexity

Number of comparisons = $n(n-1)/2$.

Time Complexity = $O(n^2)$.

Space Complexity

$O(1)$ excluding input storage.

Performance Analysis

The algorithm compares every pair of locations. As the dataset grows, comparisons increase quadratically, causing slower execution for large datasets.

Why Performance Degrades?

- Every new delivery point must be compared with all previous points.
- The number of comparisons increases quadratically.
- For small datasets, execution is fast.
- For large datasets (thousands of locations), millions of distance calculations are required.
- This increases execution time and causes delays.

Result

The brute-force closest pair algorithm was successfully implemented and correctly identified the nearest pair of delivery locations. The algorithm has $O(n^2)$ time complexity.